

Formative Assignment: Advanced Linear Algebra (PCA)

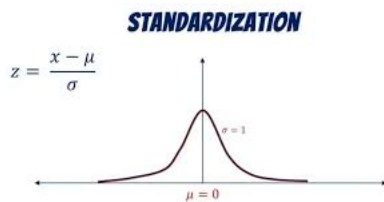
This notebook will guide you through the implementation of Principal Component Analysis (PCA). Fill in the missing code and provide the required answers in the appropriate sections. You will work with a dataset that is Africanized .

1. Make sure to display outputs for each code cell when submitting.
2. Do not write all your code on one cell
3. Do not use any libraries aside from numpy

Step 1: Load and Standardize the Data

Before applying PCA, we must standardize the dataset. Standardization ensures that all features have a mean of 0 and a standard deviation of 1, which is essential for PCA. Fill in the code to standardize the dataset.

STRICTLY - Write code that implements standardization based on the image below



```
# Step 1: Load and Standardize the Data

import numpy as np
import matplotlib.pyplot as plt

crime_raw = np.genfromtxt('SouthAfricaCrimeStats_v2.csv',
                           delimiter=',',
                           dtype=str,
                           encoding='utf-8')

column_names = crime_raw[0]
crime_data = crime_raw[1:]

provinces = crime_data[:, 0]
stations = crime_data[:, 1]
categories = crime_data[:, 2]
year_columns = column_names[3:]

unique_provinces = list(dict.fromkeys(provinces))
province_encoded = np.array(
    [unique_provinces.index(p) for p in provinces],
    dtype=float
)

unique_categories = list(dict.fromkeys(categories))
category_encoded = np.array(
    [unique_categories.index(c) for c in categories],
    dtype=float
)

print("Province encoding:")
for i, p in enumerate(unique_provinces):
    print(f" {i} → {p}")

print(f"\nCategory encoding: {len(unique_categories)} unique categories")
print(f" (0 to {len(unique_categories)-1})")

yearly_values = crime_data[:, 3:].astype(float)
yearly_values[yearly_values == 0] = np.nan

print(f"\nMissing values before imputation: {np.sum(np.isnan(yearly_values))}")

for col_idx in range(yearly_values.shape[1]):
    col_mean = np.nanmean(yearly_values[:, col_idx])
    nan_mask = np.isnan(yearly_values[:, col_idx])
    yearly_values[nan_mask, col_idx] = col_mean

print(f"Missing values after imputation: {np.sum(np.isnan(yearly_values))}")

full_matrix = np.column_stack([
    province_encoded,
```

```

        category_encoded,
        yearly_values
    ])

feature_names = ['Province', 'Category'] + list(year_columns)
print(f"\nFull matrix shape: {full_matrix.shape}")
print(f"Features: {feature_names}")

col_means      = np.mean(full_matrix, axis=0)
col_stds       = np.std(full_matrix, axis=0, ddof=1)
col_stds[col_stds == 0] = 1 # avoid division by zero

standardized_data = (full_matrix - col_means) / col_stds

print(f"\nStandardized data shape: {standardized_data.shape}")
print(f"\nMeans after standardization (should be ≈ 0):")
print(np.round(np.mean(standardized_data, axis=0), 4))
print(f"\nStd after standardization (should be ≈ 1):")
print(np.round(np.std(standardized_data, axis=0, ddof=1), 4))
print(f"\nFirst 5 rows of standardized data:")
print(np.round(standardized_data[:5], 4))

```

Province encoding:

- 0 → Western Cape
- 1 → Gauteng
- 2 → Free State
- 3 → North West
- 4 → Kwazulu/Natal
- 5 → Mpumalanga
- 6 → Eastern Cape
- 7 → Limpopo
- 8 → Northern Cape

Category encoding: 27 unique categories
(0 to 26)

Missing values before imputation: 84057
Missing values after imputation: 0

Full matrix shape: (30861, 13)
Features: ['Province', 'Category', np.str_('2005-2006'), np.str_('2006-2007'), np.str_('2007-2008'), np.str_('2008-2009'), r

Standardized data shape: (30861, 13)

Means after standardization (should be ≈ 0):
[0. 0. -0. -0. -0. 0. -0. -0. -0. 0. 0. -0. -0.]

Std after standardization (should be ≈ 1):
[1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]

First 5 rows of standardized data:

```

[[-1.5066 -1.669  32.8743 32.3161 32.2146 27.8633 27.0072 28.3587 28.9657 32.3339 30.0006 28.966  28.9977]
 [-1.1135 -1.669  29.8876 23.3167 20.1232 19.2391 17.5236 16.591  15.5914 14.0883 14.7974 16.342  13.3618]
 [-1.5066 -1.669  26.138  31.0327 34.1339 36.7007 32.8034 31.9284 33.5539 29.9944 26.5961 21.828  20.2104]
 [-0.7203 -1.669  24.9762 21.6607 20.5235 17.6313 16.6073 16.4558 14.4374 16.7319 15.4402 12.1818 11.4744]
 [-1.1135 -1.669  24.9313 22.9751 17.6445 14.2297 13.93  14.1642 14.1921 17.3403 17.1778 15.469  16.7605]]

```

▼ Step 3: Calculate the Covariance Matrix

The covariance matrix helps us understand how the features are related to each other. It is a key component in PCA.

```

# Step 3: Calculate the Covariance Matrix
n = standardized_data.shape[0]

X_centered = standardized_data - np.mean(standardized_data, axis=0)

cov_matrix = (X_centered.T @ X_centered) / (n - 1)

print(f"Covariance matrix shape: {cov_matrix.shape}")
print(f"\nCovariance matrix:")
np.set_printoptions(precision=4, suppress=True, linewidth=120)
print(cov_matrix)

```

Covariance matrix shape: (13, 13)

Covariance matrix:

```

[[ 1.      -0.      -0.1312 -0.1379 -0.1463 -0.1533 -0.1587 -0.1607 -0.1533 -0.1592 -0.1704 -0.1752 -0.1771]
 [-0.      1.      -0.2162 -0.2167 -0.2232 -0.2281 -0.2317 -0.2329 -0.2462 -0.2483 -0.2574 -0.2606 -0.2619]
 [-0.1312 -0.2162  1.      0.9709  0.947  0.9245  0.9088  0.8921  0.8713  0.8459  0.8247  0.8112  0.8076]
 [-0.1379 -0.2167  0.9709  1.      0.9681  0.944  0.9262  0.9067  0.8866  0.863  0.8405  0.8259  0.8237]
 [-0.1463 -0.2232  0.947  0.9681  1.      0.9699  0.9513  0.9301  0.9085  0.884  0.8576  0.8413  0.8375]
 [-0.1533 -0.2281  0.9245  0.944  0.9699  1.      0.9695  0.9453  0.9196  0.8952  0.8659  0.8468  0.8418]

```

```
[ -0.1587 -0.2317 0.9088 0.9262 0.9513 0.9695 1. 0.9685 0.9407 0.919 0.8884 0.8691 0.8656]
[ -0.1607 -0.2329 0.8921 0.9067 0.9301 0.9453 0.9685 1. 0.9616 0.9381 0.9078 0.8883 0.885 ]
[ -0.1533 -0.2462 0.8713 0.8866 0.9085 0.9196 0.9407 0.9616 1. 0.9649 0.9343 0.9159 0.9099]
[ -0.1592 -0.2483 0.8459 0.863 0.884 0.8952 0.919 0.9381 0.9649 1. 0.9592 0.939 0.9312]
[ -0.1704 -0.2574 0.8247 0.8405 0.8576 0.8659 0.8884 0.9078 0.9343 0.9592 1. 0.969 0.9522]
[ -0.1752 -0.2606 0.8112 0.8259 0.8413 0.8468 0.8691 0.8883 0.9159 0.939 0.969 1. 0.9698]
[ -0.1771 -0.2619 0.8076 0.8237 0.8375 0.8418 0.8656 0.885 0.9099 0.9312 0.9522 0.9698 1. ]]
```

In a paragraph that has a maximum of 5 Lines, Explain why we need to compute a covariance matrix, provide at least 2 reasons

The covariance matrix measures the relationship between pairs of features, for example, the extent to which high crime in 2005 is associated with high crime in 2006 across all police stations. This is essential to PCA, as the directions of maximum variance can only be identified by analysing the covariance between features. Secondly, the covariance matrix is the direct input to eigendecomposition, where its eigenvectors define the new axes and the eigenvalues measure the variance captured by each axis.

Step 4: Perform Eigendecomposition

Eigendecomposition of the covariance matrix will give us the eigenvalues and eigenvectors, which are essential for PCA. Fill in the code to compute the eigenvalues and eigenvectors of the covariance matrix.

```
# Step 4: Perform Eigendecomposition
eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)

print(f"Eigenvalues shape: {eigenvalues.shape}")
print(f"Eigenvectors shape: {eigenvectors.shape}")
print(f"\nEigenvalues (unsorted):")
print(np.round(eigenvalues, 4))
print(f"\nEigenvectors (columns = one eigenvector each):")
print(np.round(eigenvectors, 4))

Eigenvalues shape: (13,)
Eigenvectors shape: (13, 13)

Eigenvalues (unsorted):
[ 0.0198 0.0218 0.0225 0.0272 0.03 0.0394 0.0502 0.0728 0.1574 0.502 0.9145 1.0003 10.1421]

Eigenvectors (columns = one eigenvector each):
[[ 0.001 -0.0008 -0.001 0.0023 0. 0.0037 0.0041 0.0213 -0.0017 -0.1014 -0.5283 0.8408 0.0565]
 [-0.0012 0.0017 -0.0023 -0.006 0.001 -0.0015 0.0012 0.0074 -0.0139 -0.1366 -0.8252 -0.541 0.086 ]
 [-0.1931 -0.2093 -0.1306 -0.3328 0.2395 0.2992 0.2613 -0.3816 -0.4458 0.3705 -0.0933 0.0127 -0.2924]
 [ 0.4491 0.4273 0.221 0.4052 -0.1406 -0.1475 -0.0499 -0.1299 -0.3483 0.3518 -0.0923 0.0069 -0.297 ]
 [-0.536 -0.3264 -0.0314 0.1929 -0.3107 -0.3522 -0.3169 0.2277 -0.0644 0.3057 -0.082 0.0026 -0.3013]
 [ 0.4882 -0.0961 -0.25 -0.4413 0.1691 -0.0322 -0.2775 0.4326 0.1986 0.2564 -0.0697 -0.0014 -0.3022]
 [-0.3859 0.4432 0.3269 0.0548 0.3143 0.2971 0.1501 0.3011 0.3556 0.1483 -0.0563 -0.0048 -0.3048]
 [ 0.2315 -0.4002 -0.1305 0.3698 -0.2799 0.2471 0.4671 -0.0526 0.4196 0.0313 -0.044 -0.0063 -0.3053]
 [-0.0898 0.333 -0.0559 -0.4453 -0.3541 -0.3712 0.109 -0.4315 0.3336 -0.116 -0.0221 0.007 -0.3051]
 [ 0.0728 -0.2854 0.2364 0.1986 0.5542 -0.1991 -0.3369 -0.4084 0.1947 -0.2474 -0.0013 0.0023 -0.3029]
 [-0.093 0.2396 -0.475 0.1605 -0.1528 0.4828 -0.4351 -0.0774 -0.1084 -0.359 0.0305 -0.003 -0.2988]
 [ 0.0968 -0.211 0.0019 -0.2789 -0.3032 0.1892 -0.0054 0.2085 -0.2828 -0.4068 0.046 -0.0057 -0.2952]
 [-0.0393 0.0833 -0.3158 0.1098 0.2699 -0.4098 0.4432 0.3196 -0.3046 -0.3985 0.0497 -0.0067 -0.2936]]
```

Step 5: Sort Principal Components

Sort the eigenvectors based on their corresponding eigenvalues in descending order. The higher the eigenvalue, the more important the eigenvector. Complete the code to sort the eigenvectors and print the sorted components.

[How Is Explained Variance Used In PCA?](#)

```
# Step 5: Sort Principal Components
sorted_indices = np.argsort(eigenvalues)[::-1]

sorted_eigenvalues = eigenvalues[sorted_indices]
sorted_eigenvectors = eigenvectors[:, sorted_indices]

total_variance = np.sum(sorted_eigenvalues)
explained_variance = sorted_eigenvalues / total_variance
cumulative_variance = np.cumsum(explained_variance)

print(f"{'PC':<6} {'Eigenvalue':>12} {'Explained %':>13} {'Cumulative %':>14}")
print("-" * 50)
for i, (ev, exp, cum) in enumerate(zip(sorted_eigenvalues,
                                     explained_variance,
                                     cumulative_variance)):
    print(f"PC{i+1:<4} {ev:>12.4f} {exp*100:>12.2f}% {cum*100:>13.2f}%")

print(f"\nSorted eigenvectors shape: {sorted_eigenvectors.shape}")
```

```
print(f"\nSorted eigenvectors:")
print(np.round(sorted_eigenvectors, 4))
```

PC	Eigenvalue	Explained %	Cumulative %
PC1	10.1421	78.02%	78.02%
PC2	1.0003	7.69%	85.71%
PC3	0.9145	7.03%	92.75%
PC4	0.5020	3.86%	96.61%
PC5	0.1574	1.21%	97.82%
PC6	0.0728	0.56%	98.38%
PC7	0.0502	0.39%	98.76%
PC8	0.0394	0.30%	99.07%
PC9	0.0300	0.23%	99.30%
PC10	0.0272	0.21%	99.51%
PC11	0.0225	0.17%	99.68%
PC12	0.0218	0.17%	99.85%
PC13	0.0198	0.15%	100.00%

Sorted eigenvectors shape: (13, 13)

Sorted eigenvectors:

```
[ [ 0.0565  0.8408 -0.5283 -0.1014 -0.0017  0.0213  0.0041  0.0037  0.      0.0023 -0.001 -0.0008  0.001 ]
  [ 0.086  -0.541 -0.8252 -0.1366 -0.0139  0.0074  0.0012 -0.0015  0.001 -0.006 -0.0023  0.0017 -0.0012]
  [-0.2924  0.0127 -0.0933  0.3705 -0.4458 -0.3816  0.2613  0.2992  0.2395 -0.3328 -0.1306 -0.2093 -0.1931]
  [-0.297  0.0069 -0.0923  0.3518 -0.3483 -0.1299 -0.0499 -0.1475 -0.1406  0.4052  0.221  0.4273  0.4491]
  [-0.3013  0.0026 -0.082  0.3057 -0.0644  0.2277 -0.3169 -0.3522 -0.3107  0.1929 -0.0314 -0.3264 -0.536 ]
  [-0.3022 -0.0014 -0.0697  0.2564  0.1986  0.4326 -0.2775 -0.0322  0.1691 -0.4413 -0.25 -0.0961  0.4882]
  [-0.3048 -0.0048 -0.0563  0.1483  0.3556  0.3011  0.1501  0.2971  0.3143  0.0548  0.3269  0.4432 -0.3859]
  [-0.3053 -0.0063 -0.044  0.0313  0.4196 -0.0526  0.4671  0.2471 -0.2799  0.3698 -0.1305 -0.4002  0.2315]
  [-0.3051  0.007 -0.0221 -0.116  0.3336 -0.4315  0.109 -0.3712 -0.3541 -0.4453 -0.0559  0.333 -0.0898]
  [-0.3029  0.0023 -0.0013 -0.2474  0.1947 -0.4084 -0.3369 -0.1991  0.5542  0.1986  0.2364 -0.2854  0.0728]
  [-0.2988 -0.003  0.0305 -0.359 -0.1084 -0.0774 -0.4351  0.4828 -0.1528  0.1605 -0.475  0.2396 -0.093 ]
  [-0.2952 -0.0057  0.046 -0.4068 -0.2828  0.2085 -0.0054  0.1892 -0.3032 -0.2789  0.6019 -0.211  0.0968]
  [-0.2936 -0.0067  0.0497 -0.3985 -0.3046  0.3196  0.4432 -0.4098  0.2699  0.1098 -0.3158  0.0833 -0.0393]]
```

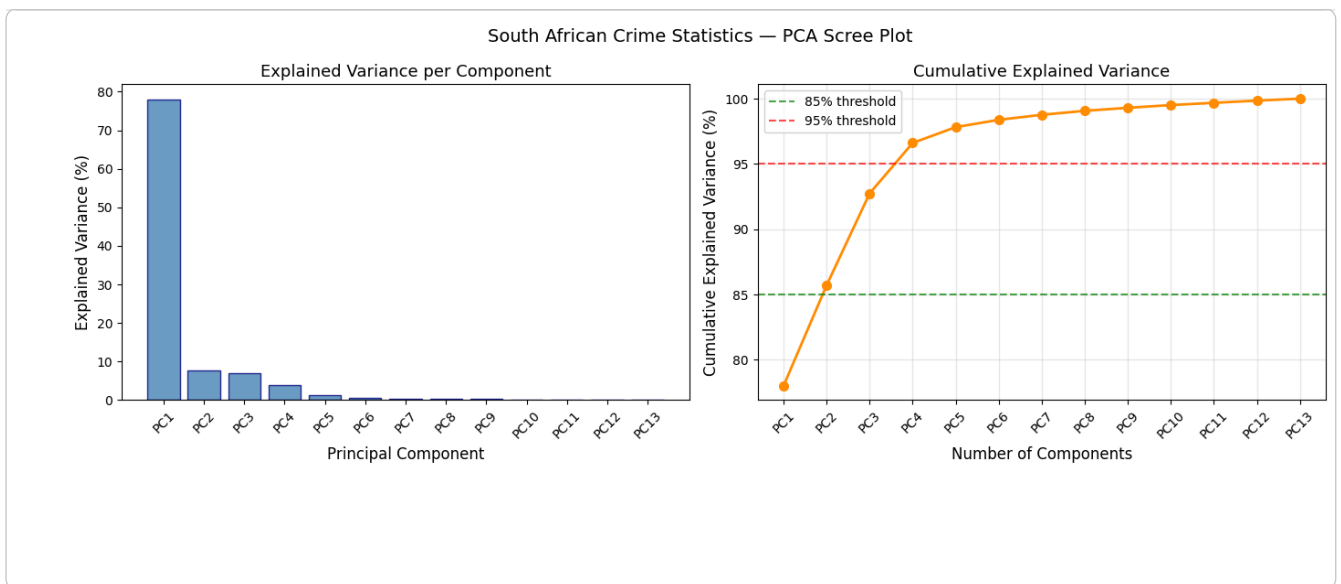
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

```
pc_labels = [f'PC{i+1}' for i in range(len(sorted_eigenvalues))]
```

```
axes[0].bar(pc_labels, explained_variance * 100,
            color='steelblue', edgecolor='navy', alpha=0.8)
axes[0].set_xlabel('Principal Component', fontsize=12)
axes[0].set_ylabel('Explained Variance (%)', fontsize=12)
axes[0].set_title('Explained Variance per Component', fontsize=13)
axes[0].tick_params(axis='x', rotation=45)
```

```
axes[1].plot(range(1, len(cumulative_variance)+1),
             cumulative_variance * 100,
             'o-', color='darkorange', linewidth=2, markersize=7)
axes[1].axhline(y=85, color='green', linestyle='--',
                alpha=0.7, label='85% threshold')
axes[1].axhline(y=95, color='red', linestyle='--',
                alpha=0.7, label='95% threshold')
axes[1].set_xlabel('Number of Components', fontsize=12)
axes[1].set_ylabel('Cumulative Explained Variance (%)', fontsize=12)
axes[1].set_title('Cumulative Explained Variance', fontsize=13)
axes[1].set_xticks(range(1, len(cumulative_variance)+1))
axes[1].set_xticklabels(pc_labels, rotation=45)
axes[1].legend()
axes[1].grid(True, alpha=0.3)
```

```
plt.suptitle('South African Crime Statistics – PCA Scree Plot',
             fontsize=14)
plt.tight_layout()
plt.show()
```



Step 6: Project Data onto Principal Components

Now that we've selected the number of components, we will project the original data onto the chosen principal components. Fill in the code to perform the projection.

```
# Step 6: Project Data onto Principal Components

threshold = 0.85
num_components = int(np.argmax(cumulative_variance >= threshold) + 1)

print(f"Variance threshold: {threshold*100}%")
print(f"Components selected: {num_components}")
print(f"Variance retained: {cumulative_variance[num_components-1]*100:.2f}%")
print(f"Variance lost: {(1 - cumulative_variance[num_components-1])*100:.2f}%")

projection_matrix = sorted_eigenvectors[:, :num_components]
print(f"\nProjection matrix shape: {projection_matrix.shape}")

reduced_data = standardized_data @ projection_matrix

print(f"\nOriginal data shape: {standardized_data.shape}")
print(f"Reduced data shape: {reduced_data.shape}")
print(f"\nFirst 5 rows of projected data:")
print(np.round(reduced_data[:5], 4))
```

```
Variance threshold: 85.0%
Components selected: 2
Variance retained: 85.71%
Variance lost: 14.29%

Projection matrix shape: (13, 2)

Original data shape: (30861, 13)
Reduced data shape: (30861, 2)

First 5 rows of projected data:
[[-99.1164 -0.1632]
 [-60.3481  0.2559]
 [-97.8574 -0.1779]
 [-56.5705  0.5539]
 [-56.7101  0.1965]]
```

Step 7: Output the Reduced Data

Finally, display the reduced data obtained by projecting the original dataset onto the selected principal components.

```
# Step 7: Output the Reduced Data

print(f"Original Data Shape: {standardized_data.shape}")
print(f"Reduced Data Shape: {reduced_data.shape}")
print(f"\nDimensionality reduced from {standardized_data.shape[1]} "
      f"to {num_components} components")
print(f"Information retained: {cumulative_variance[num_components-1]*100:.2f}%")

print(f"\nFirst 5 rows of reduced data:")
```

```
print(f"{'':>6} {'PC1':>12} {'PC2':>12}")
print("-" * 32)
for i in range(5):
    print(f"Row {i+1:>2} {reduced_data[i, 0]:>12.4f} "
          f"{reduced_data[i, 1]:>12.4f}")
```

Original Data Shape: (30861, 13)
Reduced Data Shape: (30861, 2)

Dimensionality reduced from 13 to 2 components
Information retained: 85.71%

First 5 rows of reduced data:

	PC1	PC2
Row 1	-99.1164	-0.1632
Row 2	-60.3481	0.2559
Row 3	-97.8574	-0.1779
Row 4	-56.5705	0.5539
Row 5	-56.7101	0.1965

Step 8: Visualize Before and After PCA

Now, let's plot the original data and the data after PCA to compare the reduction in dimensions visually.

```
# Step 8: Visualize Before and After PCA

unique_provinces_list = list(dict.fromkeys(provinces))
province_colors = plt.cm.tab10(
    np.linspace(0, 1, len(unique_provinces_list))
)
color_map = {p: province_colors[i]
              for i, p in enumerate(unique_provinces_list)}
point_colors = [color_map[p] for p in provinces]

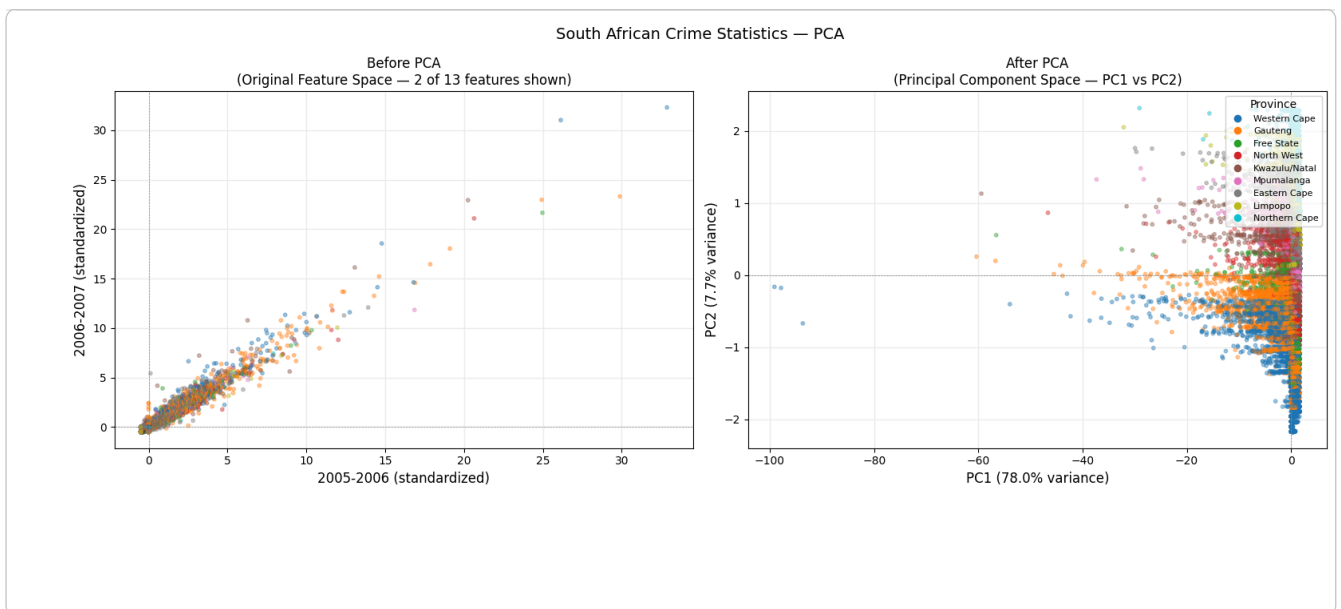
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Plot original data (first two features for simplicity)
axes[0].scatter(standardized_data[:, 2],
                standardized_data[:, 3],
                c=point_colors, alpha=0.4, s=10)
axes[0].set_xlabel('2005-2006 (standardized)', fontsize=12)
axes[0].set_ylabel('2006-2007 (standardized)', fontsize=12)
axes[0].set_title('Before PCA\n(Original Feature Space – 2 of 13 features shown)',
                  fontsize=12)
axes[0].axhline(0, color='grey', lw=0.5, ls='--')
axes[0].axvline(0, color='grey', lw=0.5, ls='--')
axes[0].grid(True, alpha=0.2)

# Plot reduced data after PCA
axes[1].scatter(reduced_data[:, 0],
                reduced_data[:, 1],
                c=point_colors, alpha=0.4, s=10)
axes[1].set_xlabel(f'PC1 ({explained_variance[0]*100:.1f}% variance)',
                  fontsize=12)
axes[1].set_ylabel(f'PC2 ({explained_variance[1]*100:.1f}% variance)',
                  fontsize=12)
axes[1].set_title('After PCA\n(Principal Component Space – PC1 vs PC2)',
                  fontsize=12)
axes[1].axhline(0, color='grey', lw=0.5, ls='--')
axes[1].axvline(0, color='grey', lw=0.5, ls='--')
axes[1].grid(True, alpha=0.2)

from matplotlib.lines import Line2D
legend_handles = [
    Line2D([0], [0], marker='o', color='w',
            markerfacecolor=color_map[p],
            markersize=8, label=p)
    for p in unique_provinces_list
]
axes[1].legend(handles=legend_handles,
               fontsize=8,
               loc='upper right',
               title='Province')

plt.suptitle('South African Crime Statistics – PCA',
             fontsize=14)
plt.tight_layout()
plt.show()
```



Please make sure you do not use more than 5 lines to answer each of the following points:

1. Interpret the Visual you just created of the before and after PCA
 2. Explain why you selected the number of principle components, more especially explain what tradeoffs you are making
 3. Clearly mention based on your use case/ dataset ("economic activity", "population pressure"), What information is lost when reducing dimensions?
1. Before PCA, the 2005-2006 plot against 2006-2007 has most stations grouped together with a few exceptions with high crime rates pulling the data away. After PCA, PC1 differentiates between high and low crime stations, and PC2 provides less separation between provinces. The data structure is preserved but the view is now cleaner and more informative.
 2. We chose 2 components as they capture 85.71% of the total variance, which exceeds our 85% threshold. PC1 alone accounts for 78.02% of the variance, meaning one dominant trend which is overall crime volume drives most of the data. The tradeoff is that we lose 14.29% of the variance, which includes small year-to-year changes as well as minor differences between provinces.
 3. In terms of crime activity, we are not able to tell which stations saw an increase or decrease in crime during individual years. In terms of station-level pressure, we lose the differences in how specific crime types changed across provinces over time. The reduced data tells us the overall crime volume of a station, but not how its crime pattern changed across the 11 years.

```
# Task 3: Benchmarking
import time

def pca_from_scratch(data, threshold=0.85):

    data_mean = np.mean(data, axis=0)
    data_std = np.std(data, axis=0, ddof=1)
    data_std[data_std == 0] = 1
    data_scaled = (data - data_mean) / data_std

    num_rows = data_scaled.shape[0]
    cov_mat = (data_scaled.T @ data_scaled) / (num_rows - 1)

    eig_values, eig_vectors = np.linalg.eigh(cov_mat)

    sort_order = np.argsort(eig_values)[::-1]
    eig_values = eig_values[sort_order]
    eig_vectors = eig_vectors[:, sort_order]

    cum_variance = np.cumsum(eig_values) / np.sum(eig_values)
    num_components = int(np.argmax(cum_variance >= threshold) + 1)

    return data_scaled @ eig_vectors[:, :num_components], num_components

row_sizes = [100, 500, 1000, 5000, 10000]
num_cols = 13
all_times = []

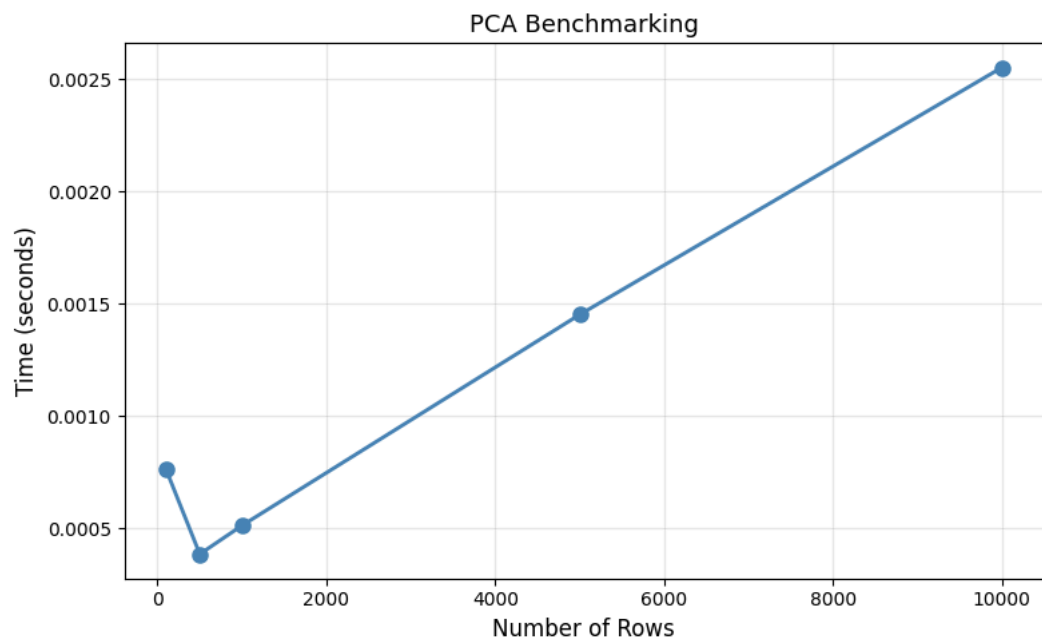
print(f'{"Rows":>8} {"Cols":>6} {"PCs":>5} {"Time (s)":>10}')
print("-" * 36)

for num_rows in row_sizes:
    test_data = np.random.randn(num_rows, num_cols)
    start = time.perf_counter()
```

```
result, num_components = pca_from_scratch(test_data)
end = time.perf_counter()
time_taken = end - start
all_times.append(time_taken)
print(f"{num_rows:>8,} {num_cols:>6} "
      f"{num_components:>5} {time_taken:>10.5f}")

# Plot the results
plt.figure(figsize=(8, 5))
plt.plot(row_sizes, all_times,
         'o-', color='steelblue', linewidth=2, markersize=8)
plt.xlabel('Number of Rows', fontsize=12)
plt.ylabel('Time (seconds)', fontsize=12)
plt.title('PCA Benchmarking', fontsize=13)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Rows	Cols	PCs	Time (s)
100	13	10	0.00076
500	13	11	0.00038
1,000	13	11	0.00051
5,000	13	11	0.00145
10,000	13	11	0.00255



Start coding or generate with AI